

Cognizant Digital Nurture 5.0

ReactJS Hands-On Exercise 17

Consuming REST APIs in React

Objective

The objective of this exercise is to understand how to consume **REST APIs** in a React application using the **Fetch API**. The application retrieves user details from an online REST service and displays the user's **title**, **first name**, and **profile image**. This exercise demonstrates asynchronous programming, API integration, lifecycle methods, and state management in React.

The application uses the **componentDidMount()** lifecycle method to fetch data after the component is rendered and displays the received data dynamically on the webpage.

Theory

What is a REST API?

A **REST (Representational State Transfer) API** is a web service that allows applications to communicate over HTTP. It enables one application to request data from another application or server.

React applications commonly consume REST APIs to display dynamic content such as user information, products, weather reports, news articles, and more.

What is Fetch API?

The **Fetch API** is a modern JavaScript interface used to send HTTP requests and receive responses from web servers.

It supports asynchronous communication using **Promises** and allows React applications to retrieve data without refreshing the webpage.

Example:

```
fetch("https://api.randomuser.me/")
```

React Lifecycle Method

componentDidMount()

`componentDidMount()` is invoked immediately after a component is added to the DOM.

It is the recommended place to:

- Call REST APIs
- Load external data
- Initialize timers
- Perform asynchronous operations

Example:

```
componentDidMount() {  
  
    fetch(API_URL)  
  
        .then(response => response.json())  
  
        .then(data => {  
  
            this.setState({  
  
                person: data.results[0]  
  
            });  
  
        });  
  
    }  
}
```

React State

React State stores data that may change during the execution of the application.

In this exercise, State stores:

- User Information
- Loading Status

Whenever the API returns data, the state is updated using:

```
this.setState()
```

Updating the state automatically refreshes the UI.

Async and Await

async and await make asynchronous code easier to read and understand.

Instead of using multiple `.then()` methods, asynchronous code can be written in a sequential manner.

Example:

```
async componentDidMount() {  
  
    const response = await fetch(url);  
  
    const data = await response.json();  
  
}
```

Advantages of REST APIs

- Retrieves live data from servers.
 - Makes applications dynamic.
 - Reduces manual data entry.
 - Supports asynchronous communication.
 - Improves user experience.
 - Easy integration with third-party services.
-

Prerequisites

- Node.js
 - npm
 - Visual Studio Code
 - ReactJS
-

Software Requirements

- React
 - JavaScript (ES6)
 - HTML
 - CSS
 - Fetch API
-

Implementation Steps

Step 1

Create a React application.

```
npx create-react-app fetchuserapp
```

Step 2

Navigate to the project folder.

```
cd fetchuserapp
```

Step 3

Create the following files inside the **src** folder.

- GetUser.js
 - App.js
 - index.js
-

Step 4

Create the component state.

Initialize state variables:

- person
 - loading
-

Step 5

Implement **componentDidMount()**.

Inside this method:

- Call the REST API.
 - Fetch JSON data.
 - Store the first user object in State.
-

Step 6

Render the user details.

Display:

- Title
 - First Name
 - User Profile Image
-

Step 7

Run the application.

npm start

Project Structure

fetchuserapp

```
|— src
|   |— Getuser.js
|   |— App.js
```

```
|  ├── index.js
|  ├── index.css
|
|  ├── public
|  ├── package.json
└── node_modules
```

Source Code

Getuser.js

```
import React, { Component } from "react";

class Getuser extends Component {
  constructor() {
    super();

    this.state = {
      person: null,
      loading: true
    };
  }

  async componentDidMount() {
    const url = "https://api.randomuser.me/";

    const response = await fetch(url);

    const data = await response.json();

    this.setState({
      person: data.results[0],
      loading: false
    });
  }

  render() {
```

```
    if (this.state.loading) {
      return <h2>Loading...</h2>;
    }
    return (
      <div style={{ textAlign: "center", marginTop: "40px" }}>
        <h1>
          {this.state.person.name.title}
          {" "}
          {this.state.person.name.first}
        </h1>
        <img
          src={this.state.person.picture.large}
          alt="User"
          width="200"
          height="200"
        />
      </div>
    );
  }
}

export default Getuser;
```

App.js

```
import React from "react";
import Getuser from "../Getuser";

function App() {
  return (
    <div>
```

```
    <Getuser />

  </div>

);
}

export default App;
```

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import App from "./App";

const root = ReactDOM.createRoot(
  document.getElementById("root")
);

root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

index.css (Optional)

```
body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background-color: #f5f5f5;
}

h1 {
  color: #222;
```



```
}  
  
img{  
  border-radius: 10px;  
  margin-top: 20px;  
  box-shadow: 0 0 8px gray;  
}
```

Output Screenshot:

When the application starts, the browser displays the **Loading...** message while the REST API request is in progress.

After the data is successfully fetched from the **Random User API**, the application automatically displays the retrieved user information.

The page contains:

- User Title
- User First Name



Every time the application is refreshed, a new random user is displayed because the API returns different user details for each request. This demonstrates successful REST API integration using the **Fetch API** and the **componentDidMount()** lifecycle method.

Learning Outcome

After completing this exercise, I learned how to:

- Consume REST APIs in a React application.
- Use the **Fetch API** to retrieve data from an external server.
- Work with asynchronous programming using **async** and **await**.
- Use the **componentDidMount()** lifecycle method for API calls.
- Store API response data in the React State.
- Dynamically display user information received from a REST API.
- Improve React applications by integrating live data from external services.

Conclusion

This exercise provided practical experience in consuming REST APIs using React. The application successfully retrieved user information from the **Random User API** and displayed the user's title, first name, and profile image. By implementing the **Fetch API**, **async/await**, **componentDidMount()**, and React State, the application demonstrated how dynamic data can be integrated into React components. This exercise strengthened the understanding of API communication, asynchronous programming, lifecycle methods, and state management, which are essential concepts for developing modern React applications.